

Portfolio Stack Decision & Rationale

Option 1 — HTML + CSS + JavaScript

How I would build:

I would create the portfolio using separate HTML pages/sections, CSS for the visual design and responsive layout, and JavaScript only where small interactions are needed, such as image galleries, navigation, or simple form behavior.

Free hosting:

Vercel, GitHub Pages, or Cloudflare Pages.

Backend: No. The current portfolio does not require one.

Trade-offs:

This is the simplest option and matches my current skills. The downside is that as the portfolio grows, I may have repeated HTML across multiple pages, making larger changes more manual.

Maintainability: High. I already know HTML, CSS, and JavaScript, so I can understand and edit the code myself.

Displaying my work: Excellent. It can handle UI/UX screenshots, long case studies, GitHub links, live demos, and Figma/design work.

Option 2: React + Vite

How I would build:

I would create reusable React components for navigation, project cards, galleries, case-study sections, and other repeated elements, with Vite handling the development and build process.

Free hosting: Vercel or Netlify.

Backend: No.

Trade-offs:

React would make a growing portfolio more organized and reusable, but I would need to spend time learning React properly before relying on it for the whole portfolio. That creates additional learning and debugging work within my two-week deadline.

Maintainability: Moderate for me right now. React is useful, but because I am not yet confident with it, maintaining the portfolio would be harder than maintaining plain HTML/CSS/JS.

Displaying my work: Excellent. It can handle all my required formats.

Option 3 : Next.js

How I would build:

I would use Next.js with React to create the portfolio and use its routing and other framework features for pages and potentially more advanced functionality later.

Free hosting: Vercel.

Backend: No backend is needed currently, although Next.js supports more advanced server-side functionality if I need it later.

Trade-offs:

Next.js is the most powerful option, but that power is unnecessary for my current portfolio. It would introduce additional concepts and require me to learn both React and Next.js.

Maintainability: Low–moderate for me at this stage because I don't have enough experience with React/Next.js yet.

Displaying my work: Excellent. It can display all my screenshots, case studies, repositories, demos, and Figma work.

Pressure-test

What becomes difficult if I choose the simplest?

Nothing important breaks initially. HTML/CSS/JS can handle everything my portfolio currently needs. The main issue is that maintaining many pages and repeated components could become more manual as my portfolio grows.

What would I maintain if I choose the most powerful?

With Next.js, I would have to maintain a more complex framework structure and learn additional concepts. I would also be maintaining features that I don't currently need.

Can I finish in two weeks?

HTML/CSS/JS: Yes, most realistically.

React + Vite: Possible, but I would need to learn React while building.

Next.js: Possible but risky because it adds both React and Next.js learning.

Does each option display my work properly?

Yes. All three can display my UI/UX screenshots, long-form case studies, GitHub repositories, live demos, and Figma work. The difference is mainly **complexity and maintainability**, not the ability to showcase my work.

My decision

I would choose HTML + CSS + JavaScript with Vercel.

I considered **React + Vite**, but I didn't choose it because I am not yet confident enough with React to make it the foundation of my portfolio. I would rather use technologies I already understand and spend my two weeks improving the portfolio itself instead of learning a framework at the same time.

I also considered **Next.js**, but it is even more powerful than I currently need. Since my portfolio does not require dynamic functionality or a backend, the additional complexity would not provide enough benefit.

Can I maintain this? Yes. HTML, CSS, and JavaScript are within my current skill level, so I can make changes, fix issues, and update my projects myself.

Does it show my work well? Yes. A static portfolio can still present my UI/UX screenshots, detailed case studies, GitHub repositories, live demos, and Figma work effectively.

Backend: Not yet. My current portfolio is primarily static, so I don't need a backend at this stage.